

Analisis Restful Api Web Service Pada Sistem Informasi Barbershop

Nicodias Palasara^{*1}, Alwiudin², Ilmi Nurchasanah³, Syarif Rizaldy Maylano⁴

^{1,2,3,4}Universitas Bina Sarana Informatika; Jl. Banten No.1, Karawang 41351

^{1,2,3,4}Sistem Informasi, Fakultas Teknik & Informatika

e-mail: ^{*1}nico.ncp@bsi.ac.id, ²19221480@bsi.ac.id, ³19221470@bsi.ac.id,

⁴19220775@bsi.ac.id

Abstrak

Barbershop merupakan perusahaan yang bergerak di bidang jasa pangkas rambut bagi pria. Peralihan proses bisnis dari yang awalnya manual berubah jadi sistem layanan berbasis website membuat pengolahan data dan informasi menjadi lebih besar dan kompleks. Oleh karena itu, kebutuhan akan pengolahan dan integrasi data harus mendapatkan perhatian yang lebih besar. Dengan adanya arsitektur web service yang menjadi entitas komputasi yang dapat diakses melalui jaringan internet dapat menghubungkan berbagai jenis aplikasi di seluruh web tanpa memperdulikan bahasa pemrograman yang digunakan. Dalam penelitian ini, metode web service yang digunakan adalah RESTful API yang terbilang sangat ringan dan sederhana dibandingkan SOAP. Metode RESTful digunakan agar kedua platform saling terhubung dan saling terintegrasi. Sedangkan untuk metode pengembangan sistemnya adalah metode waterfall karena metode ini menyediakan alur kerja yang terstruktur dan linear yang sangat membantu dalam memastikan setiap tahap pengembangan sistem dilakukan dengan cermat. Hasil pengujian yang didapatkan adalah RESTful API web service telah berhasil diimplementasikan dengan hasil pengujian 4 fitur, yaitu GET, POST, PUT dan DELETE dan sistem memberikan respon sesuai dengan yang diharapkan.

Kata Kunci—RESTFUL API, Barbershop, Web Service

Abstract

Barbershop is a company engaged in barbering services for men. The shift in business processes from initially manual to website-based service systems has made data and information processing larger and more complex. Therefore, the need for data processing and integration must receive greater attention. With the existence of a web service architecture that is a computing entity that can be accessed through the internet network, it can connect various types of applications throughout the web regardless of the programming language used. In this study, the web service method used is a RESTful API which is fairly lightweight and simple compared to SOAP. The RESTful method is used to make the two platforms interconnected and integrated with each other. As for the system development method, it is the waterfall method because this method provides a structured and linear workflow that is very helpful in ensuring that each stage of system development is carried out carefully. The test results obtained are that the RESTful API web service has been successfully implemented with the results of testing 4 features, namely GET, POST, PUT and DELETE and the system provides a response as expected.

Keywords—RESTFUL API, Barbershop, Web Service

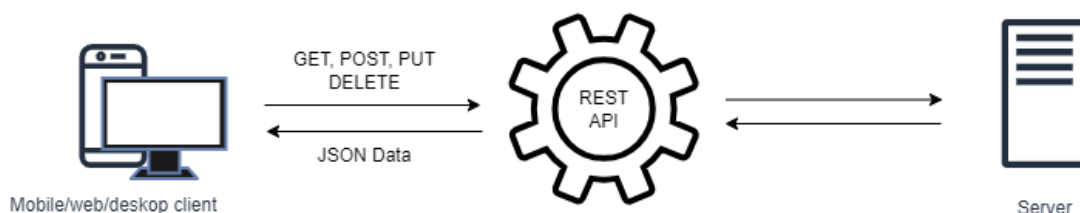
1. PENDAHULUAN

Industri bisnis jasa barbershop telah mengalami peningkatan secara signifikan seiring dengan kesadaran masyarakat akan pentingnya memperhatikan penampilan. Permintaan akan layanan yang lebih efisien menjadi tantangan di era digital saat ini untuk memberikan kepuasan kepada pelanggan. Dalam hal ini, teknologi informasi berperan sangat penting untuk memenuhi permintaan pelanggan serta segala kebutuhannya. Teknologi informasi merupakan alat bantu manusia untuk memproses kegiatan yang menghasilkan informasi [1].

Beberapa bisnis barbershop saat ini masih dilakukan secara tradisional sehingga dalam pengolahan datanya masih memiliki banyak tantangan. Kendala yang dialami bisnis barbershop ketika proses bisnisnya masih berjalan dengan sistem konvensional adalah kesulitan dalam membuat jadwal janji temu, manajemen pelanggan yang tidak efisien, serta kesulitan dalam proses pemasaran. Diperlukan sistem baru yang dapat membuat sistem tersebut menjadi sistem yang terkomputerisasi sehingga dapat meningkatkan kinerja organisasi agar dapat terus berkembang. Salah satu solusi yang bisa diterapkan adalah dengan membuat sebuah layanan barbershop dengan menggunakan teknologi *web service RESTful API*. *Web service* merupakan sebuah perangkat lunak yang tidak terpengaruh oleh *platform* serta arsitektur maupun bahasa pemrograman, yang menyediakan layanan atau metode-metode untuk pertukaran data yang dapat diakses oleh *network*. Contoh implementasinya adalah SOAP dan REST [2]. *Web service* juga memiliki fitur pengumpulan dan integrasi data yang dapat berinteraksi antar perangkat lunak [3].

Web Service menyediakan interoperabilitas antar berbagai aplikasi perangkat lunak yang *running* pada *platform* yang berbeda [4]. Data yang dibutuhkan oleh *website* dan aplikasi Android akan disediakan oleh API atau *web service*. API yang digunakan adalah metode *RESTful API* (*Representational State Transfer Application Programming Interface*) yang diciptakan oleh Roy Fielding pada tahun 2000 [5]. Istilah *RESTful API* digunakan untuk mendeskripsikan semua antarmuka sederhana yang menampilkan data menggunakan protokol HTTP tanpa *message layer* seperti SOAP atau *session tracking* dengan menggunakan HTTP *cookie* [6]. *Rest API* menggunakan protokol HTTP yang mudah diimplementasikan tanpa ketergantungan pada protokol transfer tertentu [7].

RESTful API bekerja dengan menganggap server sebagai sebuah objek yang dapat dibuat, dihapus, di *update* serta dibaca oleh *client*. Istilah yang biasanya digunakan sebagai pengkodean *RESTful API* ialah baca sebagai *request* GET, *create* sebagai *request* POST, *update* sebagai *request* PUT atau PATCH, dan hapus sebagai *request* DELETE [8]. Dan hasil dari respon yang akan dikirimkan oleh *server* ke *client* biasanya dalam bentuk format JSON dan XML sederhana tanpa ada protokol pemaketan data. XML adalah singkatan dari *eXtensible Markup Language* yang merupakan bahasa markup universal seperti HTML yang di desain untuk menyimpan dan mengantarkan data [9]. Adapun ilustrasi penggambaran *REST API Model* adalah sebagai berikut.



Gambar 1. REST API Model

Pada Gambar 1, *mobile/web/desktop* berperan sebagai *client* yang mengirimkan permintaan informasi dari layanan server. Kemudian permintaan dari *client* ke server ini akan dikirimkan menggunakan metode HTTP seperti GET, POST, PUT dan DELETE. Lalu setelah itu server akan menerima permintaan dari *client* dan memprosesnya. Setelah diproses, *server* akan memberikan *response* kembali kepada *client* yang berisi data dalam format JSON, *eXtensible Markup Language* (XML), maupun format lainnya yang telah disepakati.

RESTful API telah menjadi standar dalam komunikasi data untuk sistem berbasis web karena sifatnya yang ringan dan fleksibel. Namun, dalam beberapa tahun terakhir, muncul teknologi lain seperti GraphQL, yang menawarkan pendekatan berbeda dalam manajemen API. GraphQL muncul sebagai alternatif inovatif dengan pendekatan satu *endpoint* untuk semua operasi. Query GraphQL memungkinkan pengambilan data sesuai kebutuhan sehingga mengurangi ukuran data yang dikirim. GraphQL adalah bahasa kueri untuk implementasi API

berbasis situs web. Tidak seperti REST, GraphQL memiliki keunggulan dalam menyajikan data secara dinamis sesuai dengan aplikasi klien GraphQL menyediakan satu titik akhir untuk mengakses semua sumber data di server [10]. GraphQL menyediakan pendekatan untuk mengembangkan API web dan telah dibandingkan dan dikontraskan dengan REST dan arsitektur layanan web lainnya. Ini memungkinkan klien untuk menentukan struktur data yang diperlukan, dan struktur data yang sama dikembalikan dari server. Hal ini mencegah pengembalian data dalam jumlah yang sangat besar, tetapi dapat menghambat *caching* web dari hasil kueri. Meskipun kedua teknologi ini menawarkan keunggulan tertentu, RESTful API tetap lebih umum digunakan karena kemudahannya dalam implementasi, dukungan luas, serta kompatibilitasnya dengan berbagai infrastruktur teknologi yang ada.

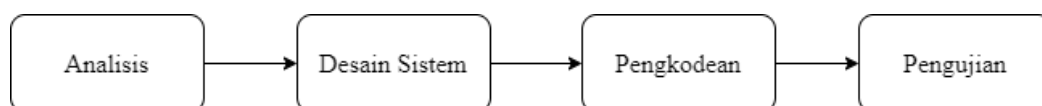
Permasalahan yang ada di *REST API* membuat permintaan data menjadi sangat lambat apalagi di saat membutuhkan data yang ingin ditampilkan dengan beberapa tabel karena *over-fetching* dan *under-fetching* yang menjadi permasalahan utama pada *REST API* [11]. Tantangan lainnya adalah dari sisi keamanan, *RESTful API* yang tidak dikonfigurasi dengan baik dapat rentan terhadap serangan seperti SQL injection dan DDoS attacks. Oleh karena itu, diperlukan mekanisme keamanan tambahan seperti autentikasi berbasis token (JWT), enkripsi data, serta firewall untuk mencegah ancaman tersebut. Selain tantangan teknis, terdapat juga kendala dalam adopsi sistem oleh pengguna. Beberapa barbershop masih mengandalkan sistem manual, sehingga diperlukan pelatihan bagi staf agar dapat beradaptasi dengan sistem digital. Faktor biaya pengembangan dan pemeliharaan sistem juga menjadi pertimbangan penting bagi bisnis kecil yang memiliki keterbatasan anggaran dalam mengadopsi teknologi ini.

Pada penelitian sebelumnya yang berjudul *Sistem Informasi Booking Online Barbershop di Mr. Grey Haircutspace Bogor* karya Ibnu Awaludin, dkk. [12] berfokus pada pengelolaan *booking online* tanpa menyoroti aspek integrasi layanan berbasis web. Dalam penelitian ini, penulis menganalisis implementasi *RESTful API* pada sistem informasi barbershop guna meningkatkan interoperabilitas, skalabilitas, dan efisiensi dalam pertukaran data antara berbagai platform. Dalam penelitian ini juga, peneliti akan membahas penerapan *RESTful API* dalam sistem informasi barbershop. Dengan menggunakan *RESTful API*, sistem informasi barbershop dapat terhubung dengan berbagai platform, seperti aplikasi *mobile*, *website*, dan sistem manajemen lainnya. Hal ini memungkinkan pemilik barbershop untuk mengelola data pelanggan, jadwal layanan, dan transaksi dengan lebih baik.

2. METODE PENELITIAN

Metode penelitian yang digunakan dalam penelitian ini adalah metode waterfall yang bersifat sistematis dan merupakan salah satu model *Software Development Lifecycle (SDLC)* yang paling sering digunakan dalam pengembangan sistem [13]. SDLC merupakan metodologi yang banyak digunakan untuk mengembangkan, memelihara, dan menggunakan sistem informasi [14]. Model *waterfall* menyediakan pendekatan alur hidup perangkat lunak secara sekuensial atau terurut, dimulai dari analisis, desain, pengkodean, dan pengujian [15]. Berikut adalah ilustrasi model *waterfall*.

2.1 Analisis

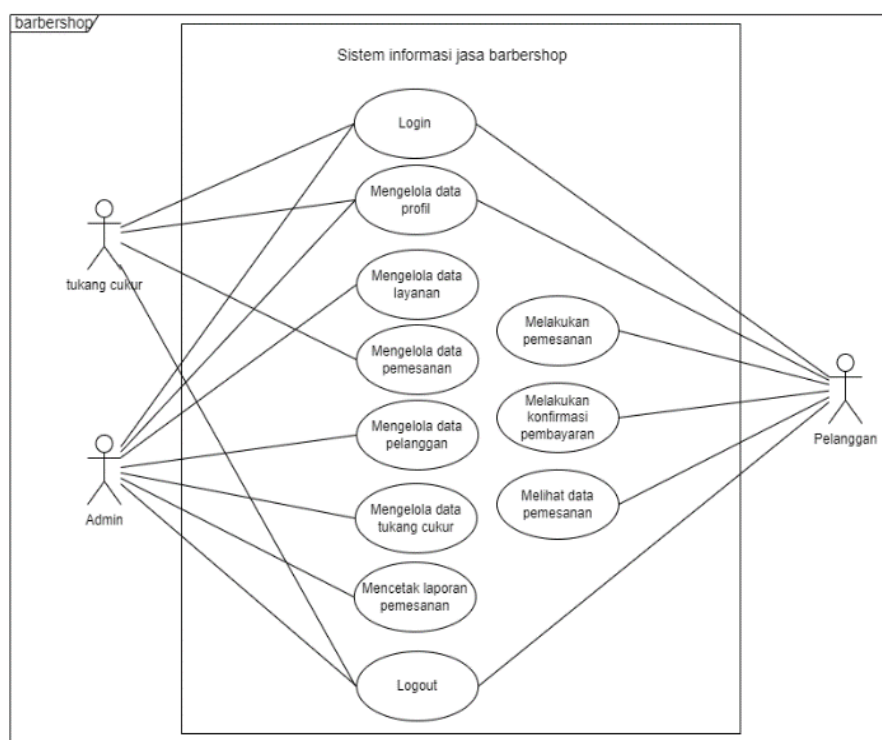


Gambar 2. Model Waterfall

Analisis kebutuhan merupakan proses untuk menentukan kebutuhan apa saja yang dibutuhkan oleh sistem. Tahapan analisis dilakukan dengan mengidentifikasi permasalahan dan membuat rancangan awal sebagai solusi dari suatu permasalahan [16].

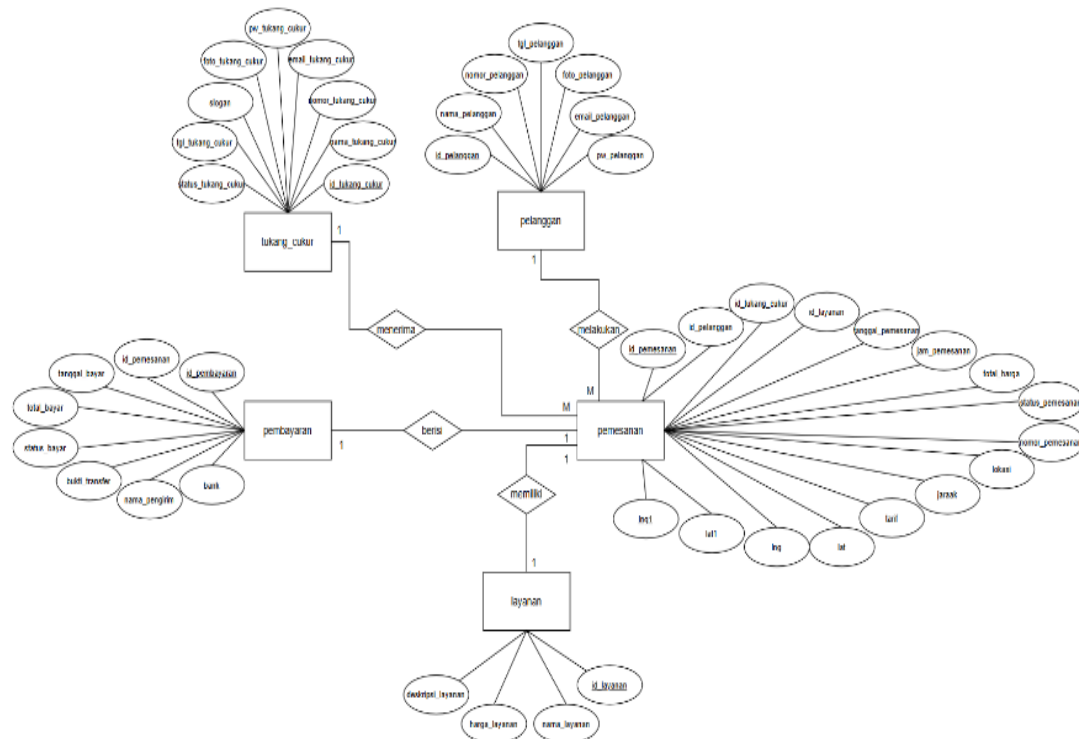
2.2 Desain Sistem

Pada tahap ini yang menjadi fokus utamanya adalah membuat desain pembuatan program perangkat lunak dengan menggunakan *Unified Modeling Language (UML)*. UML adalah salah satu alat bantu atau pemodelan yang sangat handal di dunia pengembangan sistem yang berorientasi obyek [17]. Rancangan diagram yang dibuat dalam penelitian ini adalah *use case* dan *Entity Relationship Diagram (ERD)*. Dalam pembuatan *use case*, sistem informasi barbershop memiliki tiga pengguna yang berinteraksi dengan sistem. Berikut adalah gambar *use case*.



Gambar 3. Use Case Diagram Barbershop

Entity Relationship Diagram digunakan untuk mendesain dalam basis data yang menunjukkan secara detail hubungan atau relasi antara objek dari entitas dan atributnya sehingga terbentuk secara terstruktur dan jelas menggunakan beberapa notas dan simbol [18]. Rancangan ERD dalam barbershop dapat dilihat pada gambar 4 di bawah ini.



Gambar 4. Rancangan Entity Relationship Diagram Barbershop

2.3 Pengkodean

Analisis yang telah dibuat desain sistemnya kemudian ditranslasikan ke dalam kode program sesuai dengan desain rancangan sistem yang telah dibuat sebelumnya. Pada tahap ini akan dibangun *RESTful API web service*.

2.4 Pengujian

Pada tahap pengujian ini akan dilakukan pengujian dengan menggunakan metode *Blackbox* yang akan berfokus pada persyaratan fungsional dari *web service*. Hal tersebut dilakukan untuk mendapatkan serangkaian *input* yang sepenuhnya menggunakan persyaratan fungsional untuk *web service* yang telah dirancang. Dalam menguji fungsionalitasnya dilakukan dengan alat bantu postman, yaitu dengan melakukan *HTTP request*.

3. HASIL DAN PEMBAHASAN

3.1 Pengujian dengan Postman

3.1.1 Format URI RESTful API

1. Format URI Modul Pelanggan

Tabel 1. Format URI Model Pelanggan

No	Method	Mapping
1	GET	/pelanggan /pelanggan/(id_pelanggan)
2	POST	/pelanggan
3	PUT	/pelanggan/(id_pelanggan)
4	DELETE	/pelanggan/(id_pelanggan)

2. Format URI Modul Tukang Cukur

Tabel 2. Format URI Model Tukang Cukur

No	Method	Mapping
1	GET	/ tukang_cukur /tukangcukur/(id_tukang_cukur)
2	POST	/ tukangcukur
3	PUT	/tukangcukur/(id_tukang_cukur)
4	DELETE	/tukangcukur/(id_tukang_cukur)

3. Format URI Modul Pemesanan

Tabel 3. Format URI Model Pemesanan

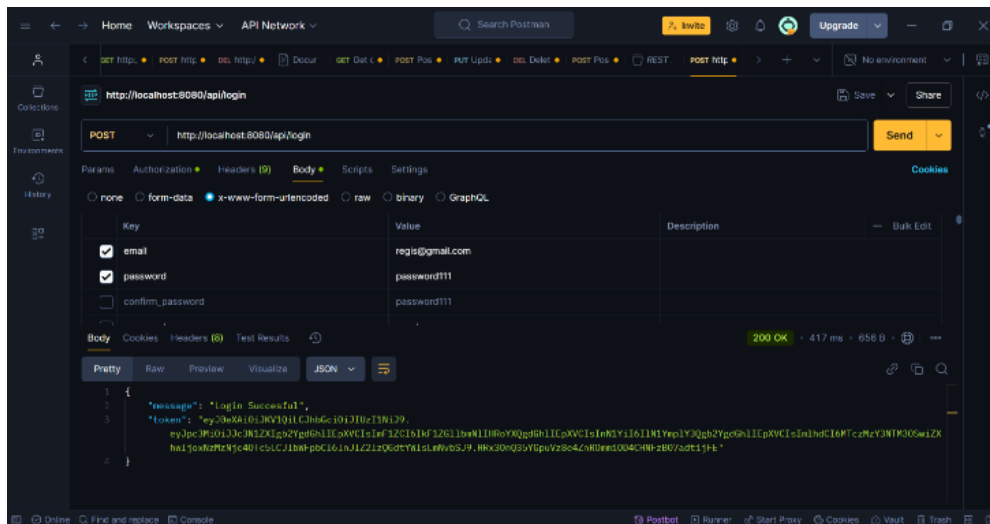
No	Method	Mapping
1	GET	/pemesanan / pemesanan /(id_pemesanan)
2	POST	/ pemesanan
3	PUT	/ pemesanan /(id_pemesanan)

4. Format URI Modul Login

Tabel 4. Format URI Model Login

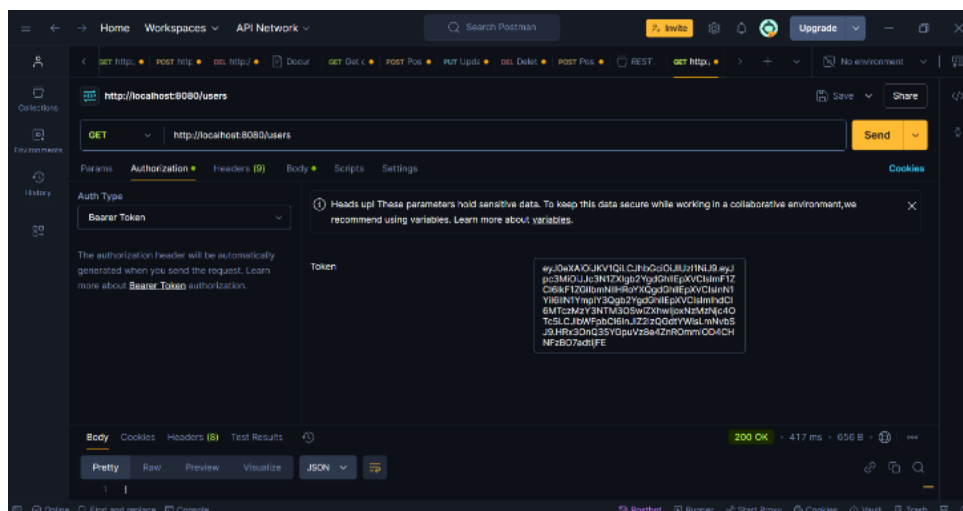
No	Method	mapping
1	POST	/api/login

3.1.2 Hasil Modul Login



Gambar 5. *Login User* untuk Mendapatkan Token

Pengujian dilakukan pada aplikasi Postman, pada alamat URI `http://localhost:8080/api/login`, dengan mengisi pada bagian *body* di kolom *x-www-urlencoded*, dengan parameter *username* dan *password*, *server* akan mengautentikasi permintaan melalui *http basic auth*, *server* lalu menggenerasikan token dan diberikan kepada *client*, dan bisa digunakan untuk *mapping-mapping* selanjutnya, dengan mengisi bagian *authorization* ganti *aunt* typenya menjadi *bearer* token kemudian isi kolom token dengan token yang sudah di beri saat *login*, seperti gambar dibawah ini:

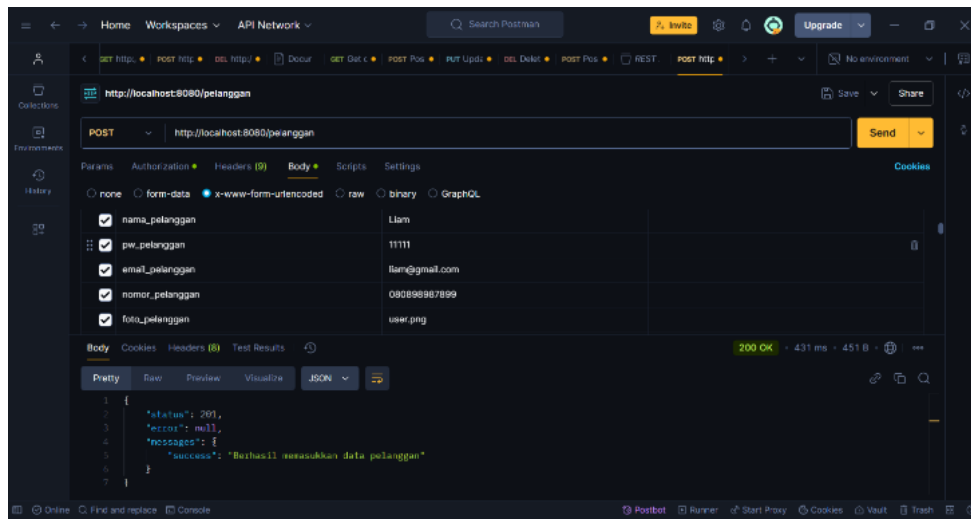


Gambar 6. Memasukkan Token pada Bearer Token

3.1.3 Hasil Modul Pelanggan

1. Melakukan *method* POST

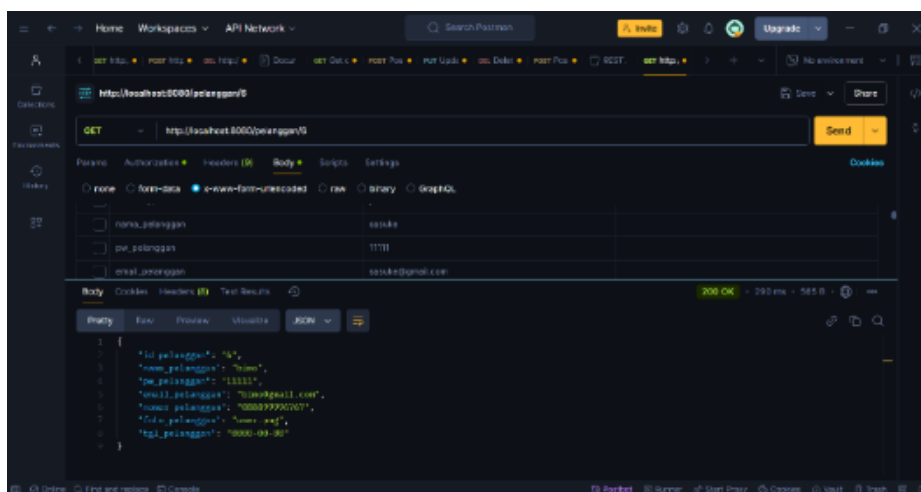
Melakukan *method* POST untuk menambahkan data pelanggan dengan URI <http://localhost:8080/pelanggan> dengan mengisi pada bagian *body* di kolom x-www-urlencoded, dengan parameter nama_pelanggan, pw_pelanggan, email_pelanggan, nomor_pelanggan dan foto_pelanggan, akan menampilkan hasil seperti pada gambar berikut ini:



Gambar 7. Hasil dari POST Pelanggan Model

2. Melakukan *method* GET

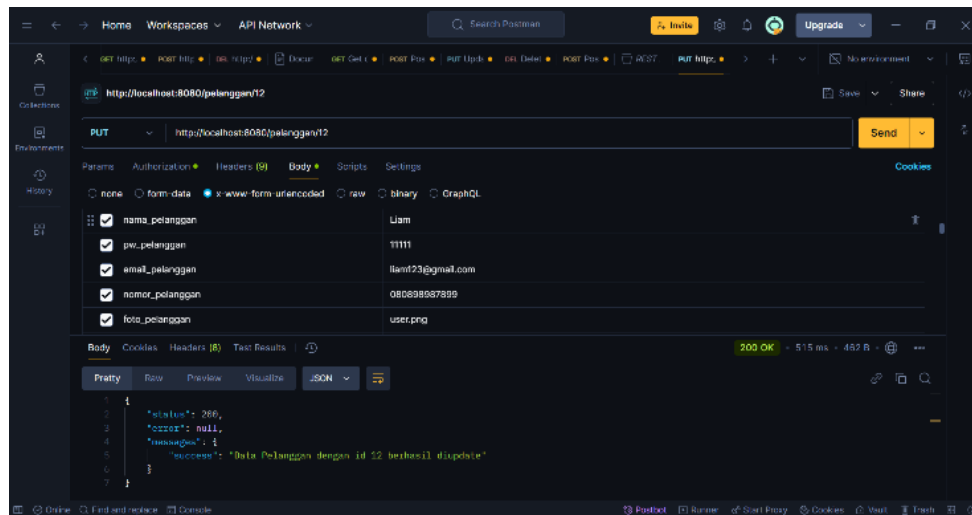
Melakukan *method* GET untuk mengecek atau melihat daftar pelanggan dengan URI [http://localhost:8080/pelanggan/\\$id](http://localhost:8080/pelanggan/$id) dengan menggantikan \$id di URI isi dengan id pelanggan, lalu dengan URI <http://localhost:8080/pelanggan> untuk melihat semua data pelanggan model yang ada pada database sistem.



Gambar 8. Hasil GET Pelanggan Model

3. Melakukan *method* PUT

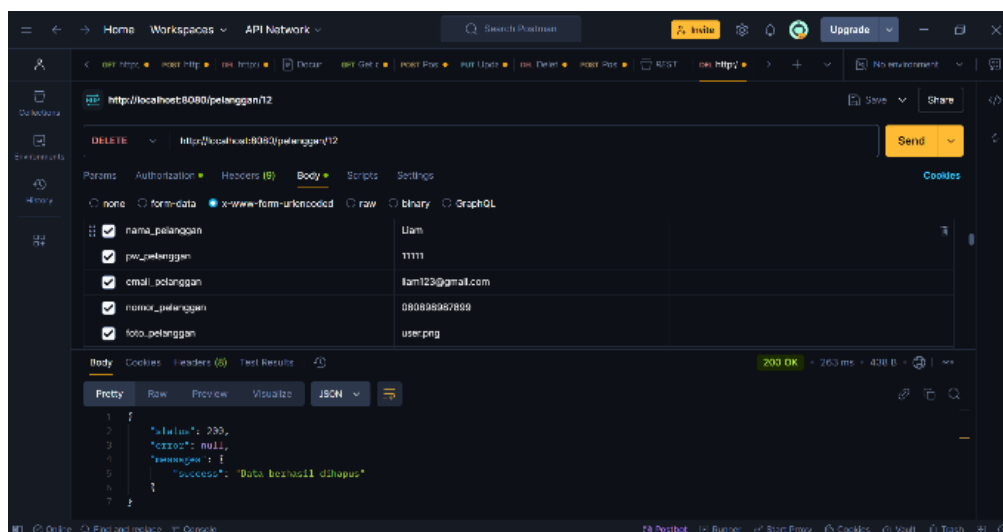
Melakukan *method* PUT untuk mengubah data pelanggan dengan URI `http://localhost:8080/pelanggan/$id` dengan mengganti URI `$id` dengan id pelanggan dan dengan mengisi pada bagian *body* di kolom `x-www-urlecodeed`, dengan parameter `nama_pelanggan`, `pw_pelanggan`, `email_pelanggan` sama seperti akan melakukan *method* POST, hanya saja ditambahkan id pelanggan pada URI.



Gambar 9. Hasil dari PUT Pelanggan Model

4. Melakukan *method* DELETE

Melakukan *delete* data dengan menggunakan *method* DELETE dengan URI `http://localhost:8080/pelanggan/$id delete` untuk menghapus data dari pelanggan model, dengan mengganti URI `$id` menjadi isi id pelanggan yang ingin dihapus seperti gambar di bawah ini.

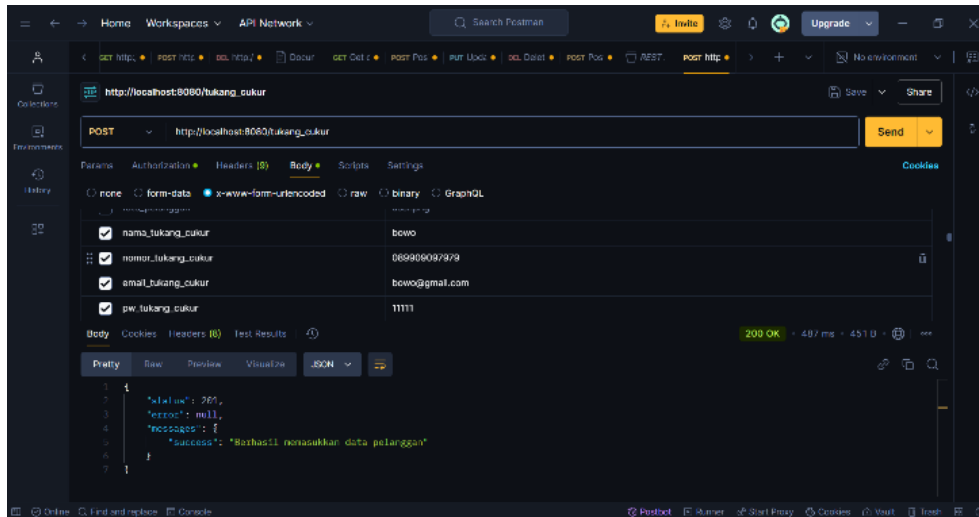


Gambar 10. Hasil dari DELETE Pelanggan Model

3.1.4 Hasil Modul Tukang Cukur

1. Melakukan *method* POST

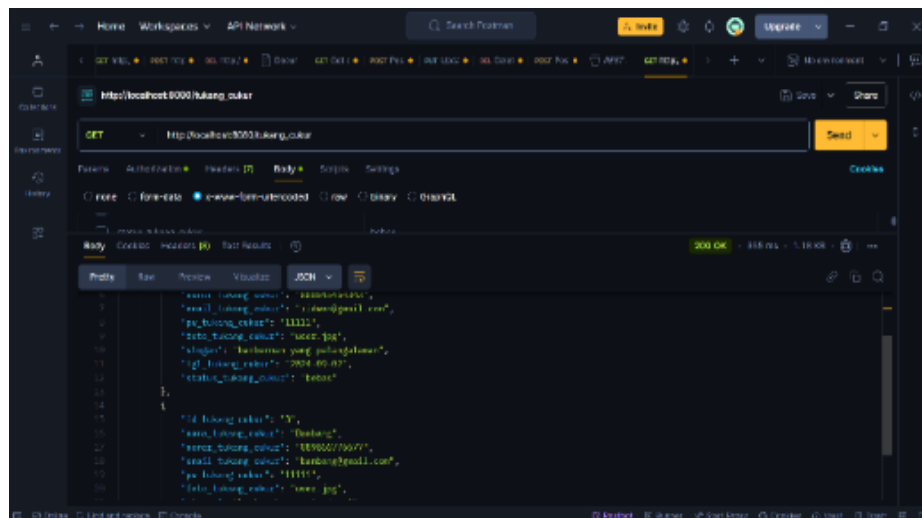
Melakukan *method* POST untuk menambahkan data tukang cukur dengan URI `http://localhost:8080/tukang_cukur` dengan mengisi pada bagian *body* di kolom *x-www-urlecoded*, dengan parameter `nama_tukang_cukur`, `pw_tukang_cukur`, `email_tukang_cukur`, `nomor_tukang_cukur`, `foto_tukang_cukur`, `status_tukang_cukur`, dan slogan akan menampilkan hasil seperti pada gambar berikut ini:



Gambar 11. Hasil dari POST Tukang Cukur Model

2. Melakukan *method* GET

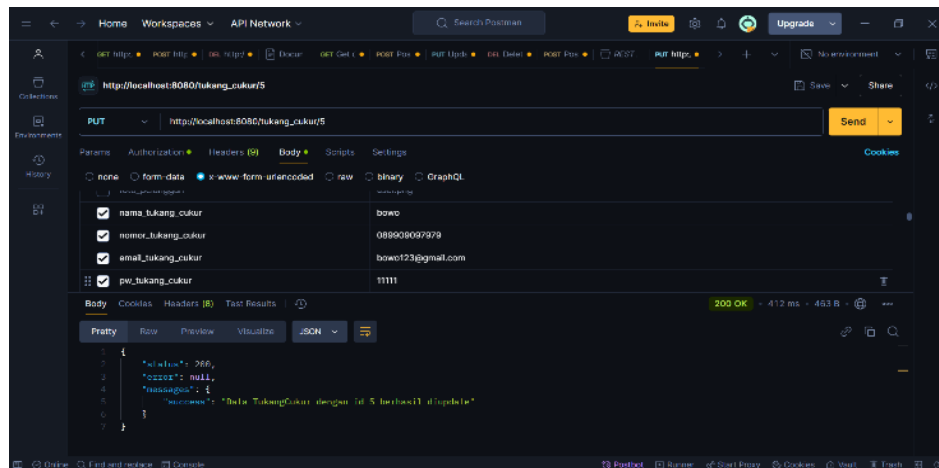
Melakukan *method* GET untuk mengecek atau melihat daftar tukang cukur dengan URI `http://localhost:8080/tukang_cukur/$id` dengan menggantikan \$id di URI isi dengan id tukang cukur, lalu dengan URI `http://localhost:8080/tukang_cukur` untuk melihat semua data tukang cukur model yang ada pada database sistem.



Gambar 12. Hasil GET Tukang Cukur

3. Melakukan *method* PUT

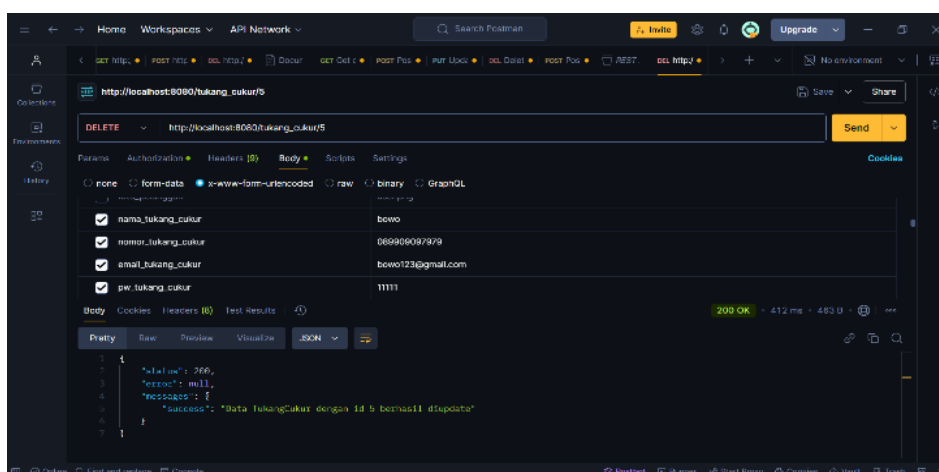
Melakukan *method* PUT untuk mengubah data tukang cukur dengan URI `http://localhost:8080/tukang_cukur/$id` dengan mengganti URI \$id dengan id tukang cukur yang ingin diubah dengan mengisi pada bagian *body* di kolom x-www-urlencoded, dengan parameter sama seperti akan melakukan *method* POST, hanya saja ditambahkan id tukang cukur pada URI.



Gambar 13. Hasil dari PUT Tukang Cukur

4. Melakukan *method* DELETE

Melakukan *delete* data dengan menggunakan *method* DELETE dengan URI `http://localhost:8080/tukang_cukur /$id delete` untuk menghapus data dari tukang cukur model, dengan mengganti URI \$id menjadi isi id tukang cukur yang ingin dihapus seperti gambar di bawah ini.

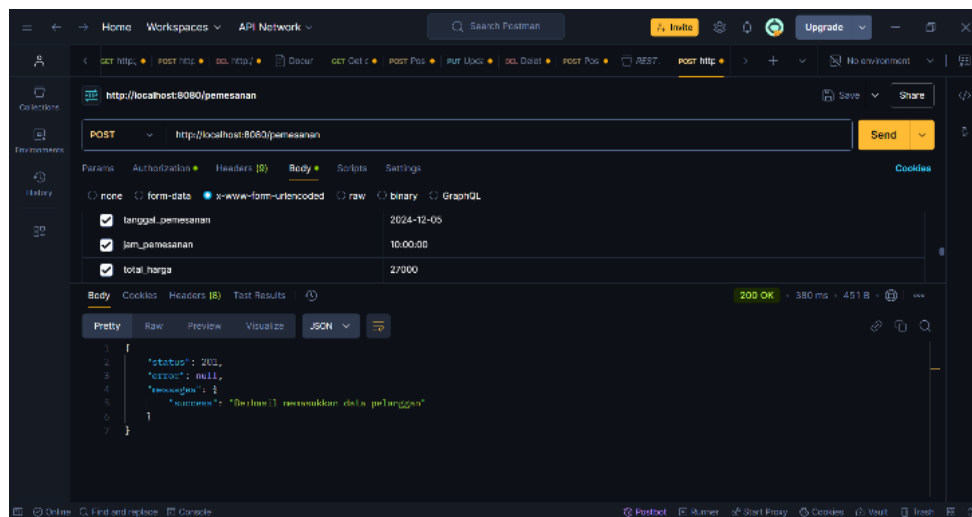


Gambar 14. Hasil dari DELETE Tukang Cukur Model

3.1.5 Hasil Modul Pemesanan

1. Melakukan *method* POST

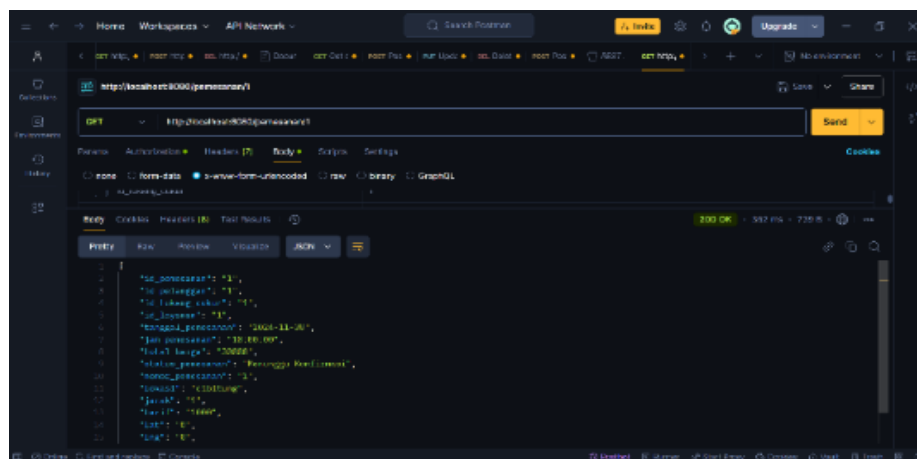
Melakukan *method* POST untuk menambahkan data pemesanan dengan URI `http://localhost:8080/pemesanan` dengan mengisi pada bagian *body* di kolom x-www-urlencoded, dengan beberapa parameter yang akan menampilkan hasil seperti pada gambar berikut ini:



Gambar 15. Hasil dari POST Pemesanan Model

2. Melakukan *method* GET

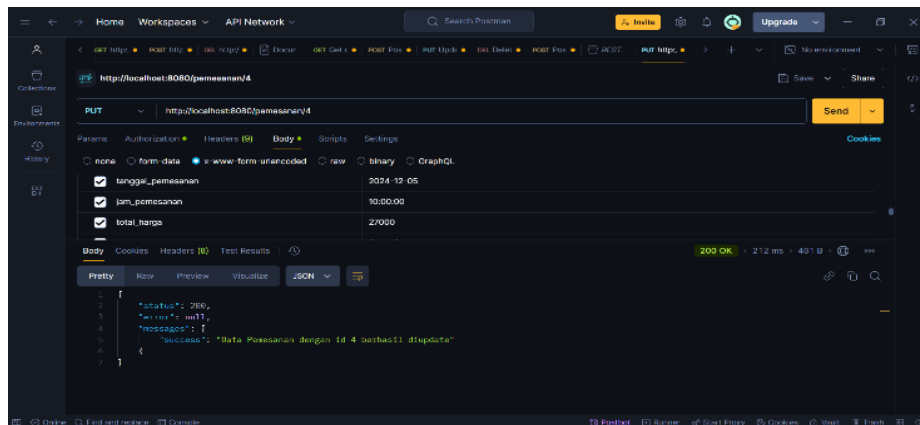
Melakukan *method* GET untuk mengecek atau melihat daftar pemesanan dengan URI `http://localhost:8080/pemesanan/$id` dengan menggantikan \$id di URI isi dengan id pemesanan, lalu dengan URI `http://localhost:8080/pemesanan` untuk melihat semua data pemesanan model yang ada pada database sistem.



Gambar 16. Hasil GET Pemesanan Model

3. Melakukan *method* PUT

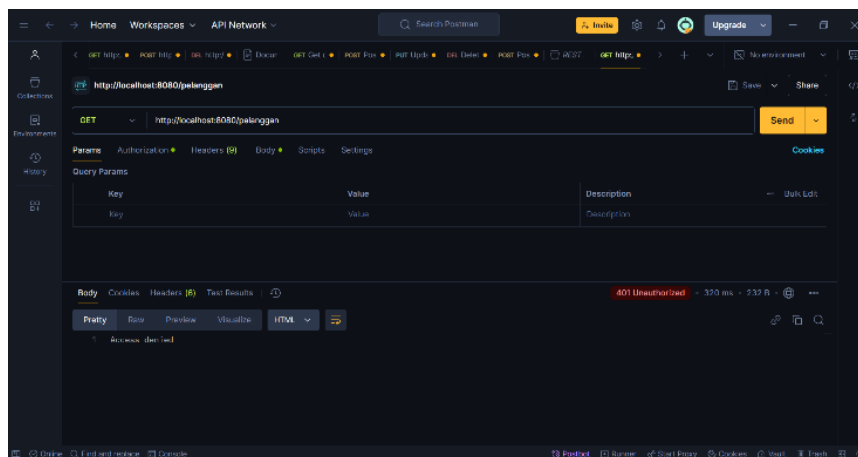
Melakukan *method* PUT untuk mengubah data pemesanan dengan URI `http://localhost:8080/pemesanan/$id` dengan mengganti URI \$id dengan id pemesanan yang ingin diubah dengan mengisi pada bagian *body* di kolom *x-www-urlencoded*, dengan parameter sama seperti akan melakukan *method* POST, hanya saja ditambahkan id pemesanan pada URI.



Gambar 17. Hasil dari PUT Pemesanan Model

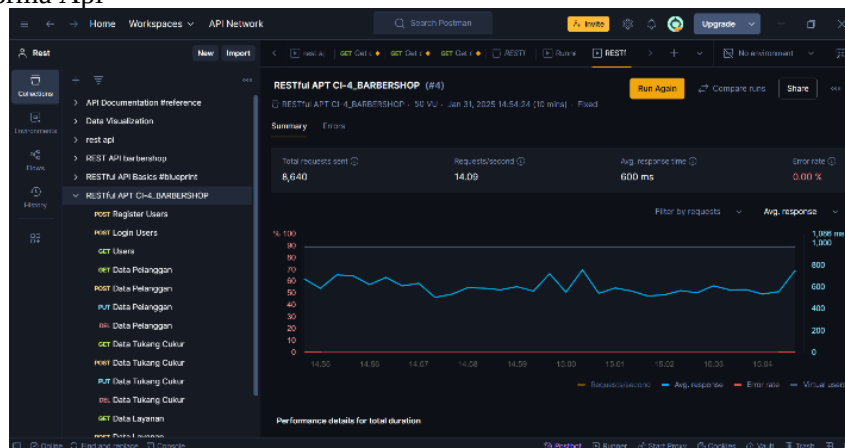
3.1.6 Hasil Pengujian Jika Tanpa Token

Setelah pengujian terhadap API berhasil dilakukan dengan token, kini saatnya menguji bagaimana API yang diakses tanpa menggunakan token pada *Authorization*, berikut adalah *output* yang ditampilkan pada aplikasi Postman saat mencoba untuk GET pada data pelanggan.



Gambar 18. Hasil dari GET Jika tanpa Token

3.1.7 Performa Api



Gambar 19. Hasil Performa Api

Pada Gambar 19, pada sistem informasi barbershop menggunakan Postman, dapat disimpulkan bahwa API memiliki tingkat kestabilan yang baik dengan rata-rata waktu respons 600 milidetik, jumlah *request* yang diproses sebesar 14 permintaan per detik, serta *error rate* sebesar 0.00%, yang menunjukkan tidak adanya kegagalan dalam menangani permintaan. Grafik performa menunjukkan bahwa waktu respons relatif stabil dengan sedikit fluktuasi, meskipun terdapat beberapa lonjakan kecil yang mungkin disebabkan oleh variasi beban server atau kondisi jaringan selama pengujian. Meskipun performa API cukup baik dalam skenario pengujian ini, masih terdapat beberapa aspek yang dapat dioptimalkan untuk meningkatkan efisiensi dan skalabilitas. Salah satu potensi perbaikan adalah reduksi waktu respons, mengingat 600 ms masih cukup tinggi untuk beberapa kasus penggunaan yang membutuhkan komunikasi cepat. Strategi seperti penerapan caching, optimasi kueri database, serta penggunaan load balancing dapat membantu mengurangi waktu respons dan meningkatkan throughput API. Selain itu, meskipun pengujian dilakukan pada tingkat 14 request per detik, diperlukan uji beban lebih lanjut dengan skenario permintaan yang lebih tinggi, seperti 100 hingga 1000 request per detik, guna mengevaluasi batas maksimal skalabilitas API dan mengidentifikasi potensi bottleneck dalam sistem.

Dari sisi keandalan, hasil pengujian menunjukkan bahwa API dapat menangani beban permintaan yang diberikan tanpa mengalami error. Namun, untuk memastikan kinerja yang tetap optimal dalam kondisi penggunaan dunia nyata, perlu dilakukan pemantauan jangka panjang terhadap performa API dengan berbagai skenario beban dan latensi jaringan yang berbeda. Dengan optimasi yang tepat, RESTful API ini dapat lebih efisien dalam menangani permintaan pengguna, meningkatkan pengalaman pelanggan, serta mendukung pengelolaan sistem informasi barbershop secara lebih optimal dan responsif.

3.2 Tampilan Website

3.2.1 Tampilan Halaman Pesanan (Pelanggan)

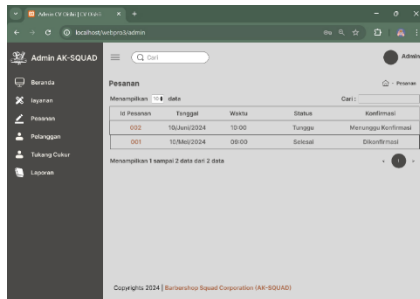
Gambar 20. Halaman Pesanan Pelanggan

Gambar 21. Halaman Form Lokasi Pelanggan

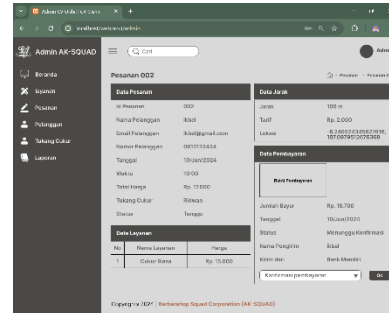
Pada gambar 20, halaman ini menampilkan form untuk data pemesanan yang harus diisi oleh pelanggan ketika melakukan pemesanan, data yang harus diisi yaitu nama, nomor hp, email, dan tanggal, waktu/jam, layanan, dan tukang cukur. Gambar 21 adalah halaman setelah selesai

mengisi form pesanan. Pada halaman ini pelanggan mengisi alamat yang sesuai dengan tempat tinggal alamat pelanggan.

3.2.2 Tampilan Halaman Pesanan (Admin)



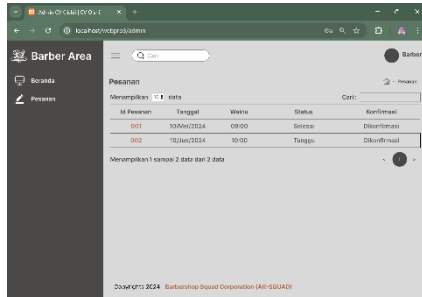
Gambar 22. Halaman Pesanan Admin



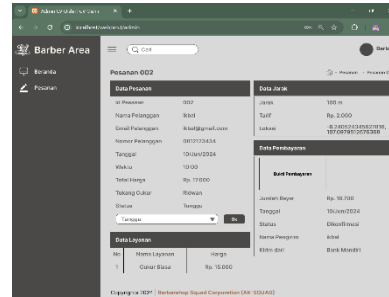
Gambar 23. Halaman Detail Pesanan Admin

Pada gambar 22 adalah halaman admin yang menampilkan informasi pesanan yang telah dilakukan oleh pelanggan. Pada tabel pesanan berisi informasi tentang id pesanan, tanggal, waktu, status, dan konfirmasi. Untuk nomor id pesanan dapat di klik untuk menampilkan detail pesannya. Gambar 23 adalah halaman admin yang menampilkan detail pesanan yang masuk dari pelanggan, admin dapat melakukan edit status pembayaran.

3.2.3 Tampilan Halaman Pesanan (Tukang Cukur)



Gambar 24. Halaman Pesanan Tukang Cukur



Gambar 25. Halaman Detail Pesanan Tukang Cukur

Pada gambar 24 adalah halaman tukang cukur yang menampilkan informasi pesanan yang telah dilakukan oleh pelanggan. Pada tabel pesanan sama seperti pesanan pada admin berisi informasi tentang id pesanan, tanggal, waktu, status, dan konfirmasi. Untuk nomor id pesanan dapat di klik untuk menampilkan detail pesannya. Gambar 25 adalah halaman tukang cukur yang menampilkan detail pesanan yang masuk dari pelanggan, berbeda dari admin di tukang cukur dapat mengedit status pesanan.

4. KESIMPULAN

Berdasarkan penelitian dan pengujian yang dilakukan dalam penelitian ini, kesimpulan yang didapat adalah penerapan *web service* dengan menggunakan arsitektur *RESTful API* pada barbershop mampu untuk diintegrasikan sehingga dapat mengatasi masalah duplikasi data. Rancang bangun *RESTful API* dilakukan dengan menggunakan metode *waterfall* melalui tahapan-

tahapan analisis, desain, pengkodean dan pengujian. Format pertukaran data menggunakan format JSON yang memudahkan server untuk diakses oleh bahasa pemrograman manapun serta sistem operasi yang berbeda. *RESTful API web service* telah berhasil diimplementasikan dengan hasil pengujian 4 fitur, yaitu GET, POST, PUT dan DELETE dan sistem memberikan respon sesuai dengan yang diharapkan.

Penelitian ini memberikan kontribusi dengan menghadirkan model implementasi RESTful API yang lebih efisien untuk sistem informasi barbershop. Dengan pendekatan ini, sistem dapat lebih mudah dikembangkan dan diintegrasikan dengan ekosistem digital yang lebih luas, berbeda dengan penelitian sebelumnya yang masih bergantung pada arsitektur monolitik, pendekatan RESTful API memungkinkan sistem untuk lebih modular, fleksibel, dan mudah dikembangkan di berbagai platform. Dengan demikian, penelitian ini memberikan landasan bagi pengembangan sistem informasi barbershop yang lebih adaptif terhadap kebutuhan industri berbasis teknologi yang terus berkembang. Meskipun penelitian ini menunjukkan efektivitas RESTful API dalam meningkatkan efisiensi sistem informasi barbershop, masih terdapat beberapa tantangan dalam implementasi dunia nyata. Faktor seperti keterbatasan infrastruktur jaringan, performa API pada skala besar, risiko keamanan, serta adopsi teknologi oleh pengguna merupakan aspek yang perlu diperhatikan untuk pengembangan lebih lanjut.

5. SARAN

Saran yang dapat diberikan oleh penulis untuk analisis *RESTful API* sistem informasi layanan *website* barbershop adalah menambahkan API yang bisa mengonfirmasi pembayaran dari halaman pelanggan tanpa mengonfirmasi lagi dari admin, dan serta menambahkan fitur-fitur yang belum tersedia untuk membantu proses transaksi agar lebih baik.

UCAPAN TERIMA KASIH

Kami selaku penulis mengucapkan terima kasih kepada Universitas Bina Sarana Informatika yang telah memberikan dukungan secara moril dan materi dalam membuat penelitian ini.

DAFTAR PUSTAKA

- [1] R. Choirudin and A. Adil, "*Implementasi Rest API Web Service Dalam Membangun Aplikasi Multiplatform Untuk Usaha Jasa*," Vol. 18, p. 284, 2019.
- [2] Edy, Ferdiansyah, W. Pramusinto, and S. Waluyo, "*Pengamanan Restful API menggunakan JWT untuk Aplikasi Sales Order*," Vol. 1, p. 107, 2021.
- [3] O. D. Arianto and Y. A. Susetyo, "*Penerapan Restful Web Service Dengan Framework Laravel Untuk Pembangunan Sistem Informasi Manajemen Sumber Daya Manusia*," Vol. 07, p. 523, 2022, doi: <https://doi.org/10.29100/jipi.v7i2.2870>.
- [4] M. I. Aulawi, S. Amini, and S. Mulyati, "*Implementasi Web Service dengan Metode Restful API dan QR Code untuk Aplikasi Manajemen Inventori pada Toko Indah Jaya Sport*," p. 211, 2022.

-
- [5] P. Painem and H. Soetanto, "Sistem Presensi Pegawai Berbasis Web Service Menggunakan Metode Restfull Dengan Keamanan JWT dan Algoritma Haversine," *Fountain Informatics J.*, Vol. 5, No. 3, p. 6, 2020, doi: 10.21111/fij.v5i3.4906.
- [6] Y. Yunhasnawa, S. Aprilianto, M. F. Allam, and D. A. S. Himawan, "Pengembangan RESTFUL API Untuk Membantu Pembudidaya Ikan Lele Dalam Sistem Manajemen Rantai Pasok 'Panen-Penen,'" p. 261, 2023.
- [7] M. I. Yanuardi, Aminudin, and M. Faiqurahman, "Analisis Perbandingan Efektivitas Arsitektur Restful dan Arsitektur Grpc pada Implementasi Web Service," *JEPIN (Jurnal Edukasi dan Penelit. Inform.)*, Vol. 10 No, p. 333, 2024.
- [8] F. Mebiyantara, A. Faisol, and F. X. Ariwibisono, "Pengembangan Aplikasi Manajemen Remote Laboratory Menggunakan Metode RESTful Web Service Berbasis Mobile," *JATI (Jurnal Mhs. Tek. Inform.)*, Vol. 5 No., no. <https://ejournal.itn.ac.id/index.php/jati/issue/view/192>, p. 154, 2021.
- [9] W. Junardi, A. P. Septiani, A. Amaliah, A. Bachtiar, and J. I. Mahendra, "Sistem Informasi Desa Siaga Pangan Menghadapi Covid19 berbasis Web Service," *J. Sist. Cerdas*, vol. Vol 03-No, no. <https://apic.id/jurnal/index.php/jsc/issue/view/8>, p. 233, 2020.
- [10] Muntahanah, Y. Darmi, and K. Pinandita, "Implementasi Perbandingan Metode GraphQL dan REST API Pada Teknologi NODEJS," *J. Inf. Technol. Comput. Sci.*, Vol. 7 No, <https://journal.ipm2kpe.or.id/index.php/INTECOM/issue/view/260>, p. 26, 2024.
- [11] F. Hanif, I. Ahmad, D. Darwis, I. L. Putra, and M. F. Ramadhani, "Analisa Perbandingan Metode GRAPHQL API dan REST API dengan Menggunakan ASP.NET Core Web API Framework," *TELEFORTECH*, Vol. 3, No, p. 33, 2022.
- [12] I. Awaludin, K. F. D. Ghazy, L. F. Simanullang, and R. Djutalov, "Sistem Informasi Booking Online Barbershop di Mr. Grey Haircutspace Bogor," *JORAPI J. Res. Publ. Innov.*, Vol. 1, No, no. <https://jurnal.portalpublikasi.id/index.php/JORAPI/issue/view/6>, p. 385, 2023.
- [13] C. Chandra, F. Wijaya, J. A. Gunawan, J. R. Lee, and A. Maulana, "Perancangan dan Implementasi RESTful API untuk Aplikasi Mobile Pembelajaran Flora dan Fauna pada Google Cloud Platform," vol. 4, p. 60, 2024.
- [14] S. Liang and V. A. Anggraini, "Perancangan Dan Implementasi Sistem Ujian di SMKN 5 Batam Menggunakan Metode SDLC," *Pros. Natl. Conf. Community Serv. Proj.*, vol. 4, pp. 1363–1368, 2022.
- [15] T. R. Ramadhini, F. Ariany, and A. Jayadi, "Sistem Informasi Presensi Karyawan Berbasis Android (Studi Kasus: Asuransi Panin Dai-Ichi Life)," *J. Teknol. Sist. Inf.*, Vol. 4, No. 1, pp. 81–88, 2023.
- [16] D. F. Kuncoro, U. Juniarti, J. Syahputra, B. R. Sumantri, Bambang Bambang, and R. Suryani, "Rancang Bangun Sistem Pengaduan Masyarakat Berbasis Web Dengan Metode Waterfall," *J. Sist. Inf. dan Teknol. Perad.*, Vol. 3, No. 2, pp. 14–19, 2022.
-

- [17] A. Voutama, "*Sistem Antrian Cucian Mobil Berbasis Website Menggunakan Konsep CRM dan Penerapan UML*," *Komputika J. Sist. Komput.*, Vol. 11 No, p. 104, 2022.
- [18] G. F. Alifi, I. P. A. P. Putra, and D. P. Agustino, "*Perancangan Sistem Informasi Penjualan dan Pengecekan Resi pada PT. Syahid Husad aDewata Berbasis Web*," *J. Tek. Inform. dan Sist. Inf.*, Vol. 11, No. <https://jurnal.mdp.ac.id/index.php/jatisi/issue/view/100>, p. 53, 2024.